



Use these Codex prompts to iteratively build the MVP; run and review each output before proceeding.

Codex prompt set for the Cambodia social-commerce MVP

Assumptions and unspecified items

UNSPECIFIED: budget, team size, launch date, domain/TLD, hosting target, SMS/OTP provider, exact payment provider credentials, exact courier API credentials, category taxonomy, fee schedule, SLA thresholds, dispute/return policy parameters, and whether you will host user-generated video in V1.

Prompt pack

1) Project folder structure and repo bootstrap

A) Codex prompt: Bootstrap a production-grade pnpm Turborepo monorepo with Next.js + TypeScript apps for `buyer`, `seller`, `admin`, and `api` (backend), plus shared packages `db` (Prisma), `core`, and `ui`; include Tailwind, ESLint/Prettier, env templates, and basic health endpoints.

B) Expected inputs:

- `NODE_VERSION` (recommend 20.x, or UNSPECIFIED)
- Package manager: `pnpm` (pin in `packageManager`)
- `APP_ENV` = `local|staging|production` (default `local`)

C) Expected outputs:

- `apps/buyer`, `apps/seller`, `apps/admin`, `apps/api` (Next.js)
- `packages/db` (Prisma setup), `packages/core`, `packages/ui`
- | | | | |
|--|-----------------------------------|--------------------------|----------------------------|
| | Root | configs: | <code>turbo.json</code> , |
| | <code>tsconfig.base.json</code> , | <code>.eslintrc</code> , | <code>.prettierrc</code> , |
| | <code>.env.example</code> , | <code>README.md</code> | |
- `apps/api/app/api/health/route.ts` (or equivalent)

D) Acceptance criteria:

- `pnpm -r lint` passes
- `pnpm -r typecheck` passes
- `pnpm -r build` succeeds
- `curl http://localhost:<api-port>/api/health` returns `{ ok: true }`

E) Sample snippet (folder skeleton):

```
apps/{api,buyer,seller,admin}/
packages/{db,core,ui}/
docker-compose.yml
.env.example
turbo.json
```

2) Prisma schema, migration, and seed

A) Codex prompt: Define Prisma schema for the marketplace core (users/roles, sellers/KYC docs, products/variants/inventory, orders/order_events, payments/payment_events, shipments, ledger/

payouts, moderation, audit) with money as `Int` minor units and enums for all statuses; add initial migration and an idempotent seed script.

B) Expected inputs:

- `DATABASE_URL` (PostgreSQL)
- Currency config: `SUPPORTED_CURRENCIES=KHR,USD` (or `UNSPECIFIED`)
- Prisma: `PRISMA_CLI_TELEMETRY_DISABLED=1` (optional)

C) Expected outputs:

- `packages/db/prisma/schema.prisma`
- `packages/db/prisma/migrations/*`
- `packages/db/prisma/seed.ts` + `db:seed` script

D) Acceptance criteria:

- `pnpm db:migrate` applies cleanly
- `pnpm db:seed` is rerunnable without duplicate rows
- `pnpm db:generate` succeeds

E) Sample snippet (OrderState enum):

```
enum OrderState {  
  CREATED  
  PAYMENT_PENDING  
  PAYMENT_CONFIRMED  
  SELLER_CONFIRMATION_PENDING  
  SELLER_CONFIRMED  
  DELIVERED  
  COMPLETED  
}
```

3) Auth (OTP phone + RBAC buyer/seller/admin)

A) Codex prompt: Implement OTP phone authentication with a pluggable provider (`UNSPECIFIED`; include a secure dev stub) and role-based access control for `BUYER`, `SELLER`, `ADMIN`; add rate limiting, session/jwt issuance, and middleware guards for API routes and UI pages.

B) Expected inputs:

- `AUTH_JWT_SECRET`
- `OTP_PROVIDER=UNSPECIFIED|devstub`
- `OTP_DEV_BYPASS=true|false`
- `REDIS_URL` (for rate limiting + OTP throttling)

C) Expected outputs:

- API endpoints: `POST /api/auth/request-otp`, `POST /api/auth/verify-otp`, `POST /api/auth/logout`
- Shared RBAC middleware in `packages/core`
- Unit tests for RBAC + rate limit behaviour

D) Acceptance criteria:

- Buyer cannot access seller/admin endpoints (403)
- OTP verify issues a session token; logout invalidates it
- Rate limit triggers 429 for repeated requests

E) Sample snippet (RBAC guard):

```
export type Role = "BUYER" | "SELLER" | "ADMIN";  
export function requireRole(userRole: Role, allowed: Role[]) {
```

```

    if (!allowed.includes(userRole)) throw new Error("FORBIDDEN");
  }

```

4) Seller onboarding + KYC upload + admin approval workflow

A) Codex prompt: Build seller onboarding with profile + payout details + KYC document upload to S3-compatible storage; implement admin approval queue, seller verification status, and risk tiering used later for payout holds.

B) Expected inputs:

- `S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_REGION` (or MINIO equivalents)
- `KYC_DOC_TYPES` (e.g., `NID`, `BUSINESS_REG`, `SELFIE`, or `UNSPECIFIED`)

C) Expected outputs:

- Prisma models: `Seller`, `SellerDocument`, `SellerBankAccount`
- Seller UI pages: onboarding wizard + status page
- Admin UI page: approvals (approve/reject + notes)
- API routes for upload (signed URLs) + approval actions

D) Acceptance criteria:

- Unapproved seller cannot create "ACTIVE" listings
- Uploaded docs stored privately; access via signed URLs only
- Admin approvals are auditable (who/when/why)

E) Sample snippet (SellerDocument model):

```

model SellerDocument {
  id          String @id @default(cuid())
  sellerId    String
  type        String
  objectKey   String
  status      String @default("PENDING")
}

```

5) Product catalog + variants + inventory + listing validation (consumer disclosure fields)

A) Codex prompt: Implement seller product catalogue with variants/SKUs, inventory, and strict listing validation including consumer disclosure fields (price/currency, seller contact channel, returns/refunds text, shipping notes); add moderation status per product and a "draft → active" publish flow.

B) Expected inputs:

- `CATEGORY_TAXONOMY=UNSPECIFIED` (create seed categories placeholder)
- Disclosure requirements: `RETURN_POLICY_TEMPLATE=UNSPECIFIED`

C) Expected outputs:

- Prisma models: `Product`, `Variant`, `InventoryItem`, `ProductMedia`
- API endpoints: product CRUD + publish/unpublish
- Validation layer (Zod) + tests

D) Acceptance criteria:

- Creating/publishing listing without required disclosure fields fails with 400 + errors
- Inventory never drops below zero under concurrent checkouts

E) Sample snippet (Zod validation fragment):

```

const ProductSchema = z.object({
  title: z.string().min(3),

```

```

currency: z.enum(["KHR", "USD"]),
returnPolicy: z.string().min(10),
sellerContact: z.string().min(5),
});

```

6) Buyer feed + product page + responsive UI

A) Codex prompt: Build buyer-facing Next.js UI (Tailwind) with a paginated discovery feed, product detail page (variants, stock, seller info, disclosures), and localisation scaffolding for Khmer/English (strings + locale switch).

B) Expected inputs:

- `DEFAULT_LOCALE=km` (or `en`, `UNSPECIFIED`)
- `SUPPORTED_LOCALES=km,en`

C) Expected outputs:

- `apps/buyer` pages/components for feed + PDP
- Data fetching hooks calling API endpoints
- Basic i18n scaffolding (no full translation required)

D) Acceptance criteria:

- Feed renders on mobile widths without layout breakage
- PDP shows disclosure fields (return policy, seller contact)
- Locale switch changes UI strings (at least 5 strings)

E) Sample snippet (feed fetch):

```

const res = await fetch(`/api/products/feed?cursor=${cursor ?? ""}`);
const { items, nextCursor } = await res.json();

```

7) Cart + checkout (COD + prepaid KHQR/PayWay adapter stubs)

A) Codex prompt: Implement cart and checkout: add/remove items, address capture, compute totals, atomically reserve inventory, create order idempotently; support `COD` and `PREPAID` with adapter stubs for KHQR and PayWay (real providers wired in step 8).

B) Expected inputs:

- `CHECKOUT_IDEMPOTENCY_TTL_SECONDS=300`
- `PAYMENT_METHODS=COD,KHQR,PAYWAY` (or `UNSPECIFIED`)

C) Expected outputs:

- API endpoints: `POST /api/cart/items`, `POST /api/checkout`
- DB transaction logic (Prisma) to reserve stock and create orders
- Tests for concurrency + idempotency

D) Acceptance criteria:

- Two concurrent checkouts for last unit results in one success, one 409
- Replaying same idempotency key returns same order id

E) Sample snippet (idempotency key usage):

```

const idemKey = req.headers.get("Idempotency-Key");
if (!idemKey) return Response.json({ error: "MISSING_KEY" }, { status:
400 });

```

8) Payment adapters + idempotent webhooks (KHQR/Bakong, ABA PayWay, ACLEDA, Wing)

A) Codex prompt: Create a `PaymentAdapter` interface and provider modules for KHQR/Bakong, ABA

PayWay, ACLEDA (Toanchet), and Wing; implement idempotent webhook handlers with signature verification hooks, store raw provider events hashed, and add a reconciliation job; if provider creds/field mappings are UNSPECIFIED, generate stubs with TODO markers and mock tests.

B) Expected inputs:

- Shared: `PAYMENTS_ENV=local|prod`, `WEBHOOK_BASE_URL`
- Bakong/KHQR: `BAKONG_API_BASE`, `BAKONG_CLIENT_ID`, `BAKONG_CLIENT_SECRET` (UNSPECIFIED ok)
- PayWay: `PAYWAY_BASE_URL`, `PAYWAY_MERCHANT_ID`, `PAYWAY_API_KEY`, `PAYWAY_WEBHOOK_SECRET`
- ACLEDA/Toanchet: `TOANCHET_BASE_URL`, `TOANCHET_MERCHANT_ID`, `TOANCHET_API_KEY`, `TOANCHET_WEBHOOK_SECRET`
- Wing: `WING_BASE_URL`, `WING_MERCHANT_ID`, `WING_API_KEY`, `WING_WEBHOOK_SECRET`
- `REDIS_URL` for queues

C) Expected outputs:

- `packages/core/payments/PaymentAdapter.ts`
- `packages/core/payments/providers/*` (4 providers)
- Webhook routes: `/api/webhooks/bakong`, `/api/webhooks/payway`, `/api/webhooks/toanchet`, `/api/webhooks/wing`
- DB: `Payment`, `PaymentEvent` (hashed payload + idempotency key)
- Tests: webhook replay idempotency + signature verification toggles

D) Acceptance criteria:

- Replaying the exact same webhook payload does not duplicate state transitions
- Webhook handler returns 2xx for already-processed events
- Failed signature returns 401 (when secrets configured)

E) Sample snippet (adapter interface):

```
export interface PaymentAdapter {
  createPaymentIntent(input: { orderId: string; amountMinor: number;
    currency: string }): Promise<any>;
  verifyWebhook(rawBody: string, headers: Record<string, string>):
    Promise<boolean>;
  parseEvent(rawBody: string): { providerEventId: string; status: string;
    orderRef?: string };
}
```

9) Strict order state machine + append-only order_events

A) Codex prompt: Implement a strict order-state machine with an explicit transition matrix enforced server-side, and an append-only `order_events` log capturing actor + metadata; include comprehensive unit tests covering every allowed/forbidden transition.

B) Expected inputs:

- `ORDER_SLA_CONFIRM_MINUTES=UNSPECIFIED`
- `ORDER_SLA_SHIP_HOURS=UNSPECIFIED`

C) Expected outputs:

- `packages/core/orders/stateMachine.ts`
- Prisma model/table `OrderEvent`
- Service methods: `transitionOrder(orderId, toState, actor)`
- Tests with full transition coverage

D) Acceptance criteria:

- Illegal transitions throw and do not mutate DB

- Every successful transition writes exactly one `order_events` row

E) Sample snippet (transition guard):

```
if (!ALLOWED[from].has(to)) {
  throw new Error(`ILLEGAL_TRANSITION:${from}->${to}`);
}
```

10) Shipping adapter + manual tracking UI + courier API stubs (J&T/Grab/Cambodia Post)

A) Codex prompt: Implement `ShippingAdapter` interface with V1 manual tracking (carrier + tracking number + status events) and V2 stub connectors for J&T, GrabExpress, and Cambodia Post; wire shipping updates to order transitions and expose buyer tracking UI.

B) Expected inputs:

- `CARRIERS=JNT,GRABEXPRESS,CAMBODIA_POST,OTHER`
- Provider creds: `JNT_API_KEY`, `GRAB_CLIENT_ID`, `GRAB_CLIENT_SECRET`, `CAMBODIA_POST_API_KEY` (UNSPECIFIED ok)

C) Expected outputs:

- Prisma models: `Shipment`, `ShipmentEvent`
- Seller UI: enter tracking + mark handed-to-carrier
- Buyer UI: tracking timeline
- Stubs: `packages/core/shipping/providers/*`

D) Acceptance criteria:

- Entering tracking moves order `SELLER_CONFIRMED/ PACKED → HANDED_TO_CARRIER` (if allowed)
- Buyer can view tracking updates for their order only

E) Sample snippet (Shipment model lines):

```
model Shipment {
  id          String @id @default(cuid())
  orderId     String @unique
  carrier     String
  trackingNo  String?
}
```

11) Ledger + payout batch service (hold windows by risk tier) + payout export

A) Codex prompt: Implement a double-entry ledger backing seller balances and platform fees; add payout batching with hold windows configured by seller risk tier, support COD (cash collected off-platform) vs prepaid settlement, and generate CSV export for manual payouts.

B) Expected inputs:

- `HOLD_DAYS_TIER_LOW=UNSPECIFIED`, `HOLD_DAYS_TIER_MED=UNSPECIFIED`, `HOLD_DAYS_TIER_HIGH=UNSPECIFIED`
- `PLATFORM_FEE_BPS=UNSPECIFIED`
- `PAYOUT_CSV_COLUMNS=default` (or UNSPECIFIED)

C) Expected outputs:

- Prisma models: `LedgerAccount`, `LedgerEntry`, `PayoutBatch`, `PayoutItem`
- Service: `postSale`, `releaseHold`, `createPayoutBatch`, `exportPayoutCSV`
- Tests: ledger balance integrity + payout reconciliation

D) Acceptance criteria:

- Ledger balances equal sum(entries) for every account in tests

- Payout totals reconcile to eligible held amounts
- COD pathway does not incorrectly “collect” customer funds in system

E) Sample snippet (ledger entry concept):

```
await createLedgerEntry({
  debitAccount: "platform_fees",
  creditAccount: `seller:${sellerId}`,
  amountMinor: feeMinor,
});
```

12) Admin/moderation console + dispute workflow + audit logs

A) Codex prompt: Build admin console to approve sellers/products, apply moderation actions, and manage disputes/refunds; implement `audit_logs` capturing actor, action, and before/after snapshots for all privileged mutations, with RBAC enforced end-to-end.

B) Expected inputs:

- `ADMIN_SEED_EMAIL=UNSPECIFIED`
- `DISPUTE_REASONS=NOT_RECEIVED,DAMAGED,NOT_AS_DESCRIBED,OTHER` (or `UNSPECIFIED`)

C) Expected outputs:

- `apps/admin` pages: approvals, moderation queue, disputes list/detail, audit log viewer
- API endpoints for moderation + dispute transitions
- Prisma model `AuditLog`, `Dispute`
- Tests: every admin mutation writes audit log

D) Acceptance criteria:

- Non-admin access returns 403 everywhere
- Audit log rows exist for approve/reject, moderation, dispute decisions

E) Sample snippet (audit log write):

```
await prisma.auditLog.create({
  data: { actorUserId, action: "SELLER_APPROVED", entityId: sellerId,
    before, after },
});
```

13) Seed data script (admin, demo sellers, bilingual products)

A) Codex prompt: Write an idempotent seed script that creates one admin, two approved sellers, one pending seller, 20 bilingual products (Khmer/English fields), variants/inventory, and a few sample orders across states for UI demos.

B) Expected inputs:

- `SEED_ENABLED=true|false`
- `SEED_ADMIN_PHONE=UNSPECIFIED` (or email if you choose email)

C) Expected outputs:

- `packages/db/prisma/seed.ts` (or `scripts/seed.ts`)
- Seed fixtures for Khmer/English text

D) Acceptance criteria:

- Running seed twice results in same counts (no duplicates)
- Buyer feed and admin dashboards show demo data immediately

E) Sample snippet (seed admin user):

```
await prisma.user.upsert({
  where: { phone: "+85500000000" },
  update: { role: "ADMIN" },
  create: { phone: "+85500000000", role: "ADMIN" },
});
```

14) Integration tests (checkout, webhook idempotency, order transitions, ledger reconciliation)

A) Codex prompt: Add integration tests (Vitest or Jest) using Testcontainers for Postgres and Redis to cover: checkout concurrency, webhook idempotency replay, order transition enforcement, and ledger/ payout reconciliation; wire tests into CI.

B) Expected inputs:

- TESTCONTAINERS_RYUK_DISABLED=UNSPECIFIED (if needed)
- CI=true

C) Expected outputs:

- tests/integration/*.test.ts
- Test utilities to spin up DB/Redis per suite
- CI workflow running tests and migrations

D) Acceptance criteria:

- Tests pass locally and in CI reliably (no flaky timing)
- Concurrency test proves no oversell

E) Sample snippet (webhook replay test idea):

```
await postWebhook(payload);
await postWebhook(payload); // replay
const events = await prisma.paymentEvent.findMany({ where: { hash } });
expect(events).toHaveLength(1);
```

15) docker-compose + local scripts + migration/runbook + CI test job

A) Codex prompt: Add docker-compose.yml for Postgres + Redis, local dev scripts (db:migrate, db:seed, db:reset), a concise migration/rollback runbook, and a CI job that lints, typechecks, migrates a test DB, and runs unit+integration tests.

B) Expected inputs:

- POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB
- DATABASE_URL, REDIS_URL in .env.local

C) Expected outputs:

- docker-compose.yml, .env.example, README.md (runbook section)
- .github/workflows/ci.yml (or equivalent)

D) Acceptance criteria:

- Fresh clone → docker compose up -d → pnpm install → pnpm db:migrate → pnpm test works
- CI passes on clean runner

E) Sample snippet (docker-compose excerpt):

```
services:
  postgres:
    image: postgres:16
```



```
redis:  
image: redis:7
```

Global acceptance checks

- **Quality gates:** `pnpm -r lint`, `pnpm -r typecheck`, `pnpm -r test` must pass at every step.
- **Security gates:** no secrets committed; `.env.example` present; webhook endpoints verify signatures when secrets configured.
- **Data integrity gates:** all money fields are integer minor units; order transitions are server-enforced; audit logs exist for privileged actions; webhook handling is idempotent.
- **Operational gates:** seed data loads; admin can approve sellers/products; a buyer can place COD and prepaid orders end-to-end in local dev.

Primary vendor docs (reference URLs)

Bakong KHQR (NBC)
<https://bakong.nbc.gov.kh/en/>
<https://bakong.nbc.gov.kh/download/>

ABA PayWay
<https://developer.payway.com.kh/>

ACLEDA Toanchet / sandbox
<https://toanchetpay.acledabank.com.kh/toanchetpay/sandbox>

Wing merchant (partner/merchant info)
<https://www.wingbank.com.kh/en/business/payments-collections/wing-merchant>

J&T API
<https://developer.jet.co.id/>

GrabExpress Business (Cambodia)
<https://www.grab.com/kh/en/business/express/>

Cambodia Post
<https://cambodiapost.com.kh/home>